

Lab 1 - RISC-V Assembly

Overview

Welcome to the world of RISC-V! As a modern, open-standard architecture, its clean and modular design has quickly made it the go-to choice for both academia and industry. In this lab, you will get hands-on experience writing RISC-V assembly (RV32I + M extension) using the Ripes visual simulator.

Please note that any form of plagiarism will result in zero marks for the entire lab. Additionally, you are required to include detailed comments explaining both your code and the underlying algorithm.

Tools and Documents:

- Simulator: Ripes (Desktop: <https://github.com/mortbopet/ripes>)
- The RISC-V Instruction Set Manual, Volume I: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>
- RISC-V Instruction Set Manual: <https://github.com/riscv/riscv-isa-manual>

Submission: Submit a compressed file **groupXX_lab1.zip** containing all the files mentioned below.

- task0_report.pdf: Report for RISC-V vs ARM
- task1.s: Array traversal and conditional logic.
- task2.s: Subroutine implementation.
- task3a.s: Matrix multiply using software multiplication.
- task3b.s: Matrix multiply using the hardware mul instruction.
- task3c.s: Matrix multiply optimized for column-major access.

Task 0: Architectural Comparison - RISC-V vs ARM

Objective: Research and analyze the fundamental architectural and philosophical differences between RISC-V and ARM, and evaluate the strategic impact of RISC-V in the modern computing landscape.

Task (task0_report.pdf): Submit a **2-page** report comparing the RISC-V and ARM architectures. Research and discuss their fundamental differences in licensing, modularity, and strategic importance in hardware-software co-design.

Task 1: Array Traversal and Conditional Branching

Objective: Write a RISC-V program using I-type and R-type instructions to traverse an array, apply conditional logic, and store results back to memory.

Task (task1.s): Given an array of 5 integers, write the assembly logic that iterates through the array. If an element is even, add 10 to it. If it is odd, subtract 5. Store the modified values back into their original memory locations.

Open the provided task1.s template. You must design and implement the loop structure and branching logic yourself in the designated area.

Verification: Run the program to completion in Ripes. Switch to the **Memory** tab and inspect the .data segment to ensure the array has been updated correctly. The original array [12, 7, 4, 15, 8] should now be [22, 2, 14, 10, 18].

Task 2 - Software Arithmetic Subroutine (RV32I Only)

Objective: Implement integer arithmetic using only the base instruction set by managing your own registers, loops, and bitwise operations.

Task (task2.s): You are strictly forbidden from using the mul instruction. Open the provided task2.s template. You must write the logic for the mymul subroutine to multiply two arguments passed in a0 and a1. Use the Shift-and-Add algorithm.

Verification: Step through the execution in Ripes to ensure the jal and ret instructions correctly manage the return address (ra) register. Once the program halts, check the **Memory** tab to ensure the calculated product is stored correctly in the result variable.

Task 3 - Matrix Multiplication Implementations

Objective: Implement a complete RISC-V program that multiplies two 4 x 4 matrices. You will produce three versions of the same algorithm to handle different instruction sets and memory layouts.

Problem Statement:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Compute $C = A \times B$, where:

$$C[i][j] = \sum_{k=0}^3 A[i][k] \cdot B[k][j]$$

Part A - Multiply Without mul (task3a.s)

- Open the provided task3a.s template, which contains the initialized matrices.
- Integrate the mymul subroutine you wrote in Task 2.
- The matrices are stored in standard row-major order.
- Use three nested loops to walk the rows and columns, calling mymul for the arithmetic.

Part B - Multiply With mul (task3b.s)

- Duplicate your completed task3a.s code.
- Remove the mymul subroutine and replace the subroutine calls with the M-extension's mul instruction.
- Ensure the structure of the three nested loops remains identical. Verify the output matrix C in memory perfectly matches Part A.

Part C - Column-Major Access (task3c.s)

- Take your task3b.s code and modify it so that matrix B is stored and accessed in **column-major order**.
- In memory, the entries of B are laid out column-by-column: B[0][0], B[1][0], B[2][0], B[3][0], B[0][1], etc.
- Re-declare matrix B in your .data section accordingly.
- Adjust the address arithmetic inside your loops for every load that touches matrix B.
- Matrices A and C remain in row-major layout. Verify the final computed C matrix is still mathematically correct in memory.